Assignment no. 2

Natural Language Processing
NLP4IL (AIS.ba VZ)
WS 2026

This assignment sheet covers the augmentation and tokenization of text. Use the following five sentences as content for all following exercises.

1. Production on the film began in June 1921 with Colleen's return from New York and Florida, where she was making The Lotus Eater with John Barrymore which was directed by Marshall Neilan.
2. The device operates as long as the magic smoke is trapped inside of it, but when the smoke escapes from it, the device ceases to operate.
3. The 1980s farm crisis caused some families to have to give up their farms, and farms have been merged to industrial scale.
4. From conceptual structuration, the architecture on which the co-simulation framework is developed and the formal semantic relations/syntactic formulation are defined.
5. The chest is a chestnut colour with thin black bars, while long black-margined plumes arise from its flanks.

Exercise 1 Simple Data Augmentation

Implement a simple data augmentation technique by randomly swapping words. Create an algorithm that swaps a specified ratio of words in the sentence. Make sure to swap the same word just once.

Task

- Generate 5 augmented sentences from the input corpus using random swaps.

Exercise 2 Synonym Replacement Augmentation Use WordNet from NLTK to replace words with synonyms. Make the ratio of word replacements configurable.

Tasks

- Retrieve synonyms for selected words
- Replace words in a sentence with synonyms
- Generate at least 3 augmented versions of the input corpus

Exercise 3 Back-Translation Augmentation

Use the transformers package to augment data by back-translation. The following provides a function to use transformers to translate. There are many models on HuggingFace, on possible option is `Helsinki-NLP/opus-mt_tiny_eng-fra`.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, AutoConfig

def roundtrip_translate(sentences, model_a_name, model_b_name):
# Load models + tokenizers
tokenizer_a = AutoTokenizer.from_pretrained(model_a_name)
model_a = AutoModelForSeq2SeqLM.from_pretrained(model_a_name)

tokenizer_b = AutoTokenizer.from_pretrained(model_b_name)
model_b = AutoModelForSeq2SeqLM.from_pretrained(model_b_name)

results = []

for text in sentences:
        inputs_a = tokenizer_a(text, return_tensors="pt")
        outputs_a = model_a.generate(**inputs_a)
        translated = tokenizer_a.decode(outputs_a[0], skip_special_tokens=True)

        inputs_b = tokenizer_b(translated, return_tensors="pt")
        outputs_b = model_b.generate(**inputs_b)
        back_translated = tokenizer_b.decode(outputs_b[0], skip_special_tokens=True)

        results.append((translated,back_translated))

return results
```

Tasks

- Augment the input corpus by backtranslation.
- Generate at least 3 augmented versions by using different languages.

Exercise 4 Basic Tokenization

Write a Python script that performs word and sentence tokenization on the input corpus using NLTK.

- Install the required libraries (`nltk`).
- Download tokenizer resources using `nltk.download()`.
- Tokenize the text

Tasks

- Perform word tokenization
- Print the resulting tokens of the input corpus

Exercise 5 Tokenization with spaCy

Use spaCy to tokenize text and analyze linguistic features.

```
import spacy
nlp = spacy.load("en_core_web_sm")

text = "Text to tokenize."
doc = nlp(text)
```

Tasks

- Print all tokens
- Print token POS tags
- Print token lemmas

As discussed in the exercise session, the following two exercises are considered bonus exercises.

Exercise 6 Comparing Tokenization Methods on IMDb (NLTK, spaCy, and BPE)

Compare three tokenization strategies for sentiment classification on the IMDb dataset on eLearning:

1. NLTK word tokenization
2. spaCy tokenization
3. The GPT 2 PBE tokenizer.

We can easily use the GTP2 PBE tokenizer with the AutoTokenizer in transformers package used in Exercise 3:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "Hello, world!"
encoded = tokenizer(text)
#Token IDs: [15496, 11, 995, 0]
#Tokens decoded as strings: ['Hello', ',', ' world', '!']
```

Each tokenizer will be used inside an identical scikit-learn pipeline with `TfidfVectorizer` and `LogisticRegression`.

Model Pipeline (example):

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

def build_pipeline(tokenizer_fn):
        return Pipeline([
                ("tfidf", TfidfVectorizer(
                tokenizer=tokenizer_fn,
                lowercase=True,
                token_pattern=None  # disable default tokenization
                )),
                ("clf", LogisticRegression(
                max_iter=200,
                solver="lbfgs"
        ))
])
```

Tasks

- Use the IMDb dataset (train/test split or provided splits).
- Prepare the tokenizers for the use with the sklearn Pipeline.
- Train and evaluate three pipelines.
- Report and compare:
  - Accuracy and F1-score on test data
  - Training time (fit time)
  - Inference time (per 1000 samples)
  - Vocabulary size from the TF–IDF layer
  - Average tokens per document for each tokenizer

Exercise 7 Implementing a Byte Pair Encoding (BPE) Tokenizer

Implement your own *Byte Pair Encoding (BPE)* tokenizer **from scratch** (no external BPE/tokenizer libraries). The tokenizer will consist of two main components:

- **Training**: learn merge rules from the IMDb training corpus
- **Tokenization**: apply merges greedily to produce subword tokens

Guidelines for Your Implementation:

1. Represent each unique word as a character sequence with an end-of-word marker, e.g. `movie → m o v i e </w>`.
2. Count symbol-pair frequencies across the full corpus vocabulary.
3. Iteratively merge the most frequent pair, updating the vocabulary after each merge.
4. Continue until you reach your target vocabulary size (e.g. 2000 merges).
5. Implement a `tokenize()` method that applies the learned merges greedily.

Tasks

- Load the IMDb dataset (train split only for BPE training).
- Implement the BPE training procedure.
- Implement the BPE tokenization procedure.
- Test the tokenizer on sample input sentences.

When using Large Language Models (eg. ChatGPT, Copilot, ...) to produce output that is handed in, also state the prompt and model used in the submission. Submissions that are not marked and are noticeably AI generated will be graded with zero points!

Submission: electronically via Moodle; Deadline: Mo, Mar 16, 2026, 23:59.